Цифровая обработка изображений

Лекция 3

Характерные дефекты изображений. Классификация фильтров.

Свертка в пространственной и частотной областях.

Сафонов Илья Владимирович

ilia.safonov@gmail.com

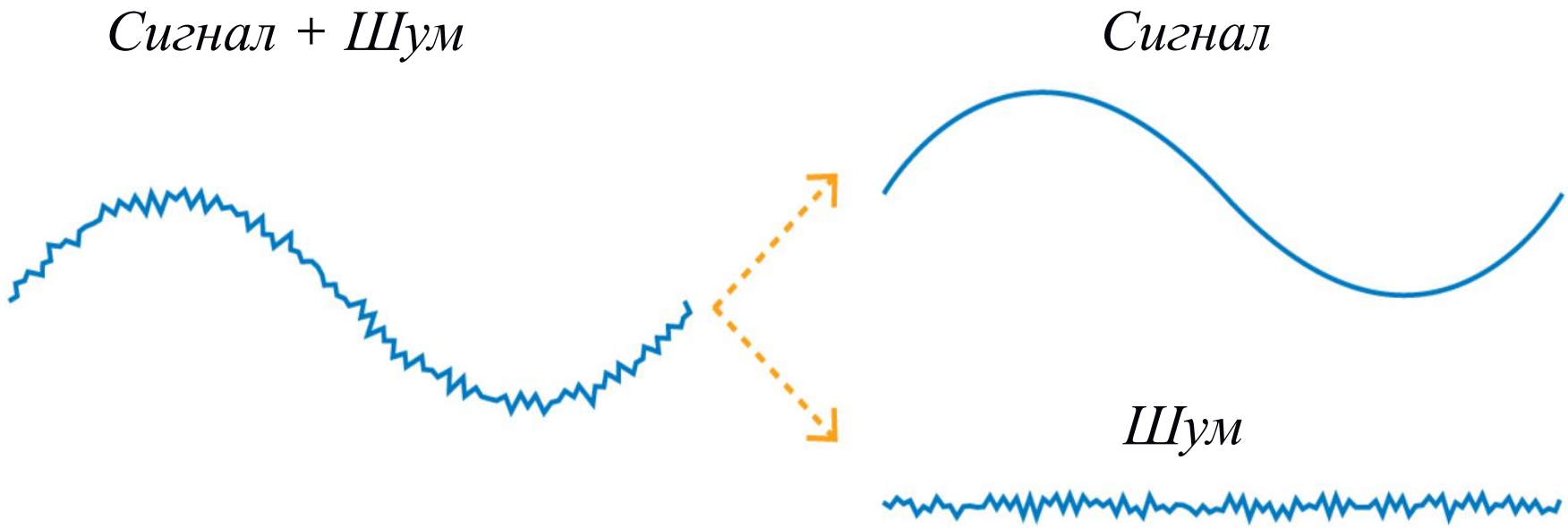
2025

Характерные дефекты изображений

- Шум
- Артефакты компрессии
- Муар
- Нечеткость/размытость
- Низкий контраст
- Искажения цветов
- Неравномерный фон

Шум

Шум (помеха) – это случайные нежелательные изменения сигнала, которые могут возникать во время его захвата, хранения, передачи и обработки.



В цифровой обработке сигналов и изображений обычно речь идёт о высокочастотных шумах, но встречается и низкочастотный шум, например, дрейф среднего значения сигнала. В обработке изображений низкочастотный шум часто называют неравномерностью фона.

Классификация шумов (1)

Существует множество вариантов классификации шумов в зависимости от области применения. В цифровой обработке сигналов и изображений для высокочастотных шумов выделяют:

• аддитивный шум (*additive noise*):

$$S'(t) = S(t) + N(p_1, \ldots, p_n),$$

где $N(p_1, \ldots, p_n)$ – модель шума, зависящая от параметров $p_1, \ldots, p_n$. Величина шумовой составляющей не зависит от значения сигнала.

• мультипликативный шум (*multiplicative noise*, *speckle*):

$$S'(t) = S(t) + S(t) \times N(p_1, \ldots, p_n)$$

Величина шумовой составляющей зависит от значения сигнала.

• импульсный шум (*impulsive, 'salt & pepper'*):

$$S'(t)=(1-p)S(t)+pN(p_1, \ldots, p_n),$$ где $p=\{0,1\}$.

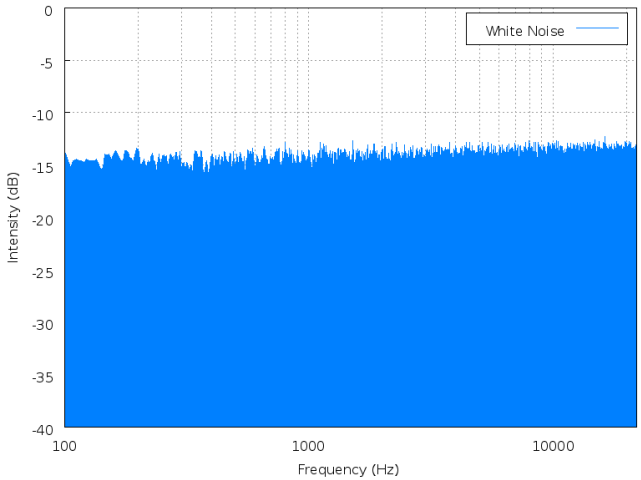
Только часть отсчётов изменяется, обычно изменённые отсчёты принимают максимальное и минимальное из допустимых значений.

Классификация шумов (2)

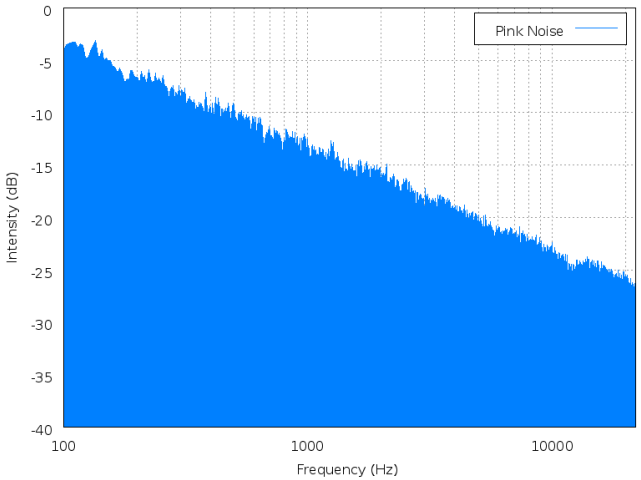
Если значение шумовой составляющей пиксела (отсчёта) зависит от значений шумовых составляющих окрестных пикселов (отсчётов), то такой шум является коррелированным, в противном случае некоррелированным (декоррелированным).

Если в амплитудном спектре спектральные составляющие некоррелированного шума распределены равномерно, то такой шум называют **белым**, в противном случае – **цветным**. В зависимости от вида амплитудного спектра шума выделяют несколько видов («цветов») цветного шума.

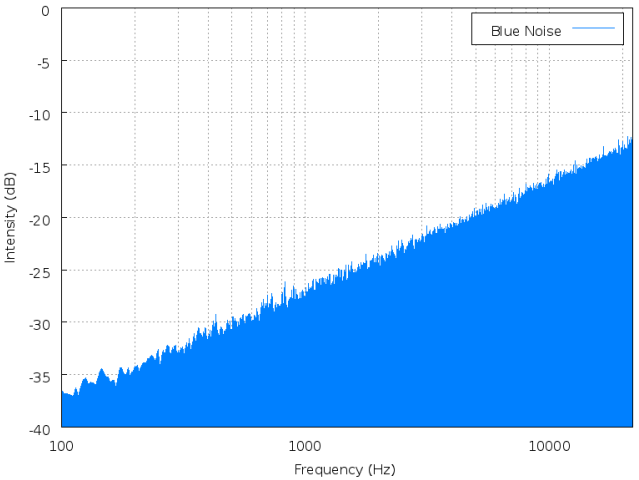
Белый шум



Розовый шум



Синий шум



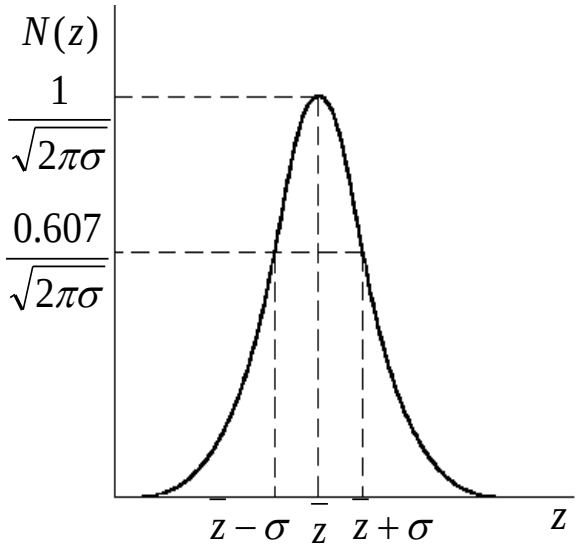
Аддитивный белый Гауссов шум

Одной из наиболее популярных моделей шума является аддитивный белый Гауссов шум (*Additive White Gaussian Noise, AWGN*)

Функция плотности распределения вероятностей для нормального распределения:

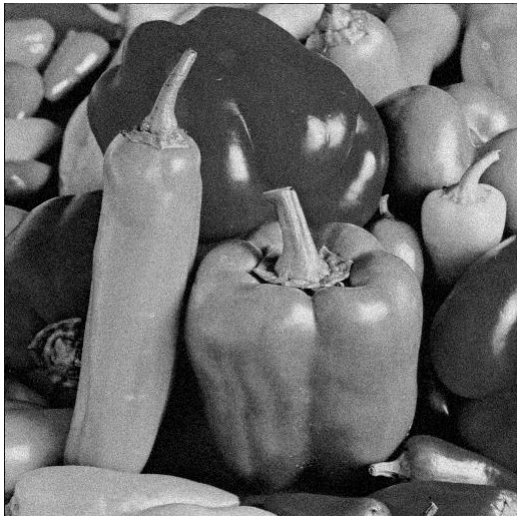
$$N(z) = \frac{1}{\sqrt{2\pi}\sigma} e^{-\frac{(z-\bar{z})^2}{2\sigma^2}}$$

В *AWGN* $\bar{z} = 0$

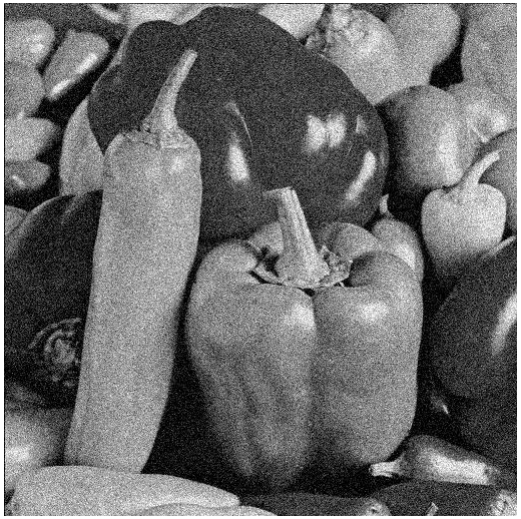


Исходное изображение

AWGN σ = 10



AWGN σ = 25



Генерация нормального распределения

Для генерации псевдослучайных чисел с нормальным распределением с заданными параметрами можно использовать алгоритм Бокса – Мюллера.

Нормальное распределение: $N\left(\bar{z}, \sigma^2\right)$

<u>Алгоритм:</u>

1. Генерируются независимые случайные величины x и y, равномерно распределённые на отрезке [-1, 1].

2. Вычисляем $s = x^2 + y^2$. Если $s > 1$ или $s = 0$, то возвращаемся на шаг 1.

3. Вычисляем: $z_0 = x\sqrt{\dfrac{-2\ln s}{s}}, \qquad z_1 = y\sqrt{\dfrac{-2\ln s}{s}}.$

4. Получившиеся z_0 и z_1 являются независимыми случайными величинами, удовлетворяющими стандартному нормальному распределению.
Для получения желаемого нормального распределения необходимо получившиеся значения умножить на σ и прибавить z.

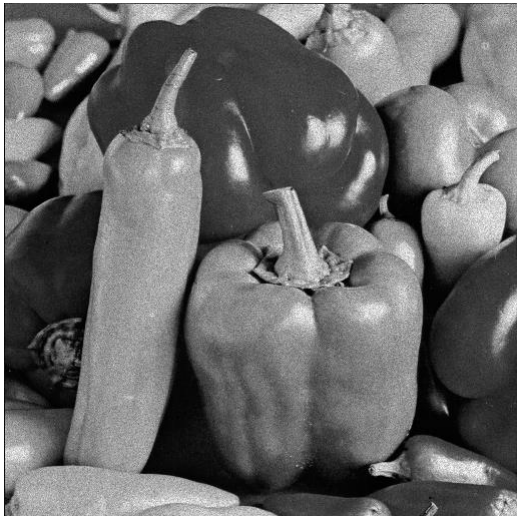
Мультипликативный белый Гауссов шум

Также используют модель мультипликативного белого Гауссова шума (*Multiplicative White Gaussian Noise*, *MWGN*).

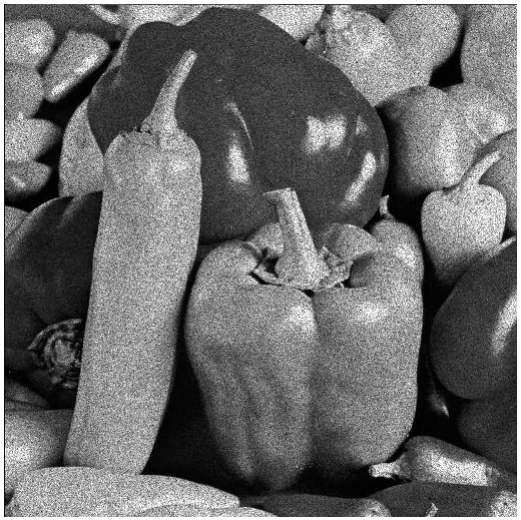
В *MWGN* $\bar{z} = 1$ $\sigma^2 = \dfrac{\sigma_k^2}{\sigma_k^2 + \bar{z}}$

Исходное изображение

MWGN $\sigma_k = 10$



MWGN $\sigma_k = 25$



Импульсный шум

Импульсный (salt and pepper) шум.

Вероятность:

$$P(z) = \begin{cases} P_a, & z = a \\ P_b, & z = b \\ 0, & иначе \end{cases}$$

Плотность распределения вероятностей:

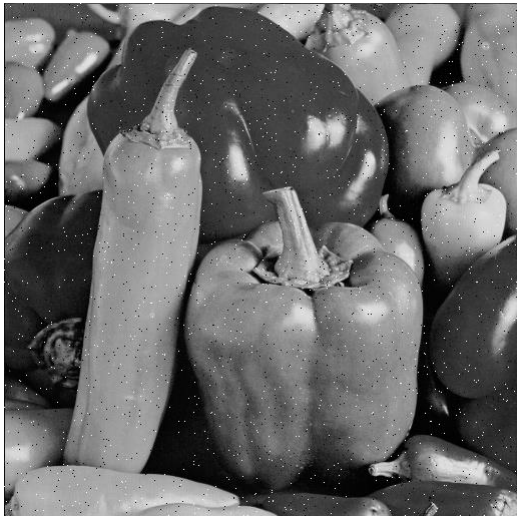
$$p(z) = P_a \delta(z-a) + P_b \delta(z-b)$$

$\delta(.)$ - дельта функция

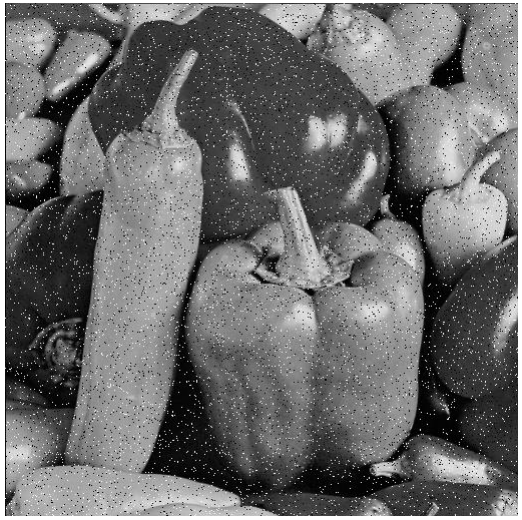
Значение каждого пиксела изображения с вероятностью P_s заменяется на случайное значение шума a или b

Исходное изображение

Ps = 0.01



Ps = 0.05



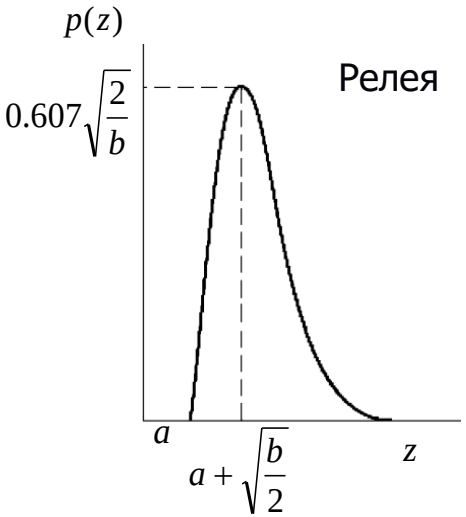
Модели шума

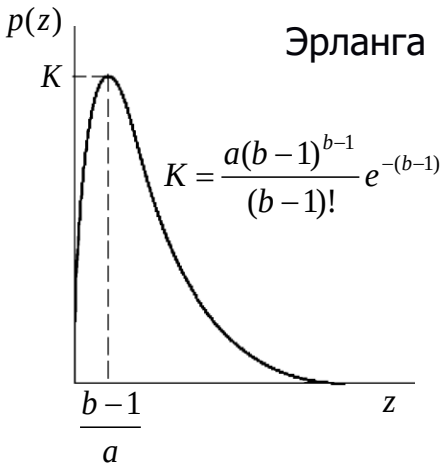
Однородный (равномерный) шум: $p(z)=\begin{cases}\dfrac{1}{b-a}, & a\le z\le b\\ 0, & иначе\end{cases}$ $\mu=\dfrac{a+b}{2}$ $\sigma^2=\dfrac{(b-a)^2}{12}$

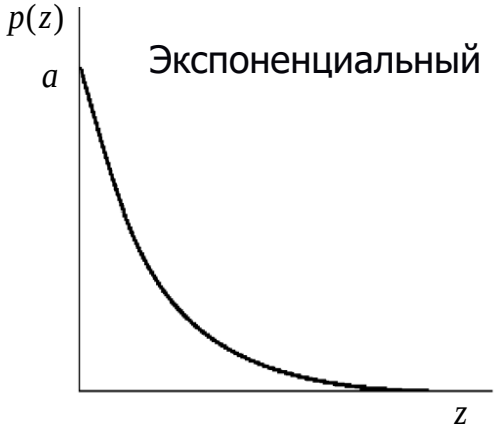
Шум Релея (Rayleigh): $p(z)=\begin{cases}\dfrac{2}{b}(z-a)e^{-(z-a)^2/b}, & z\ge a\\ 0, & z<a\end{cases}$ $\mu=a+\sqrt{\pi b/4}$ $\sigma^2=\dfrac{b(4-\pi)}{4}$

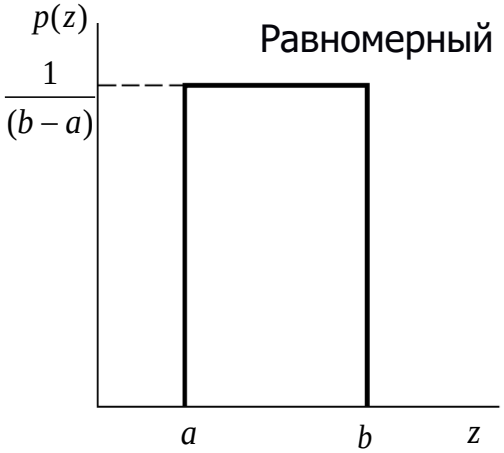
Шум Эрланга (Erlang) /гамма шум: $p(z)=\begin{cases}\dfrac{a^b z^{b-1}}{(b-1)!}e^{-az}, & z\ge 0\\ 0, & z<0\end{cases}$ $\mu=\dfrac{b}{a}$ $\sigma^2=\dfrac{b}{a^2}$

Экспоненциальный шум: $p(z)=\begin{cases}ae^{-az}, & z\ge 0\\ 0, & z<0\end{cases}$ $\mu=\dfrac{1}{a}$ $\sigma^2=\dfrac{1}{a^2}$





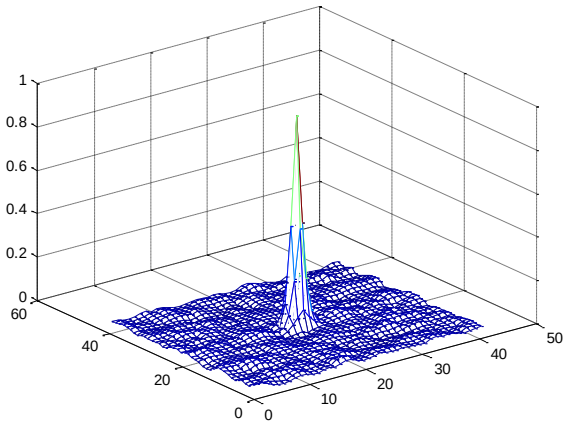




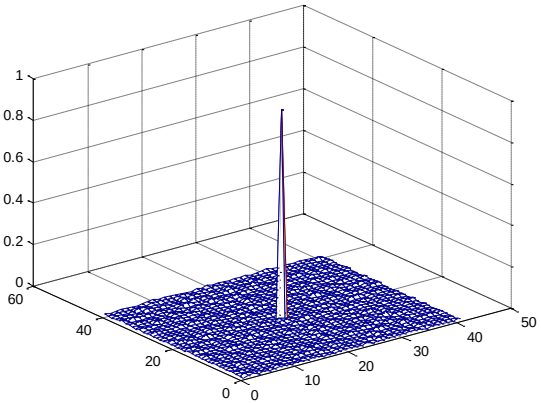
Шумы цифровых фотокамер

Шумы современных цифровых камер имеют модель более сложную, чем модели рассмотренные ранее.

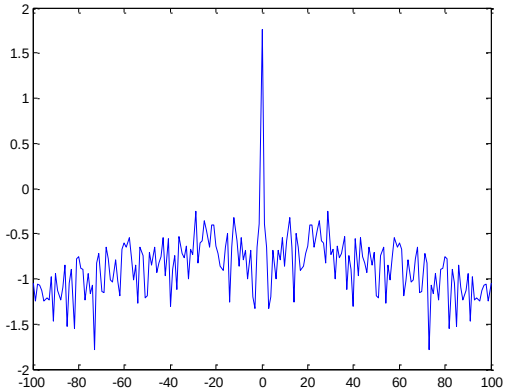
Шум цифровой камеры



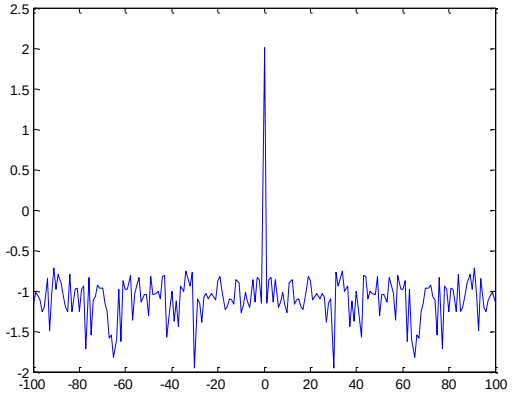
AWGN



Шум является пространственно коррелированным, т.к. автокорреляционная функция отличается от дельта функции



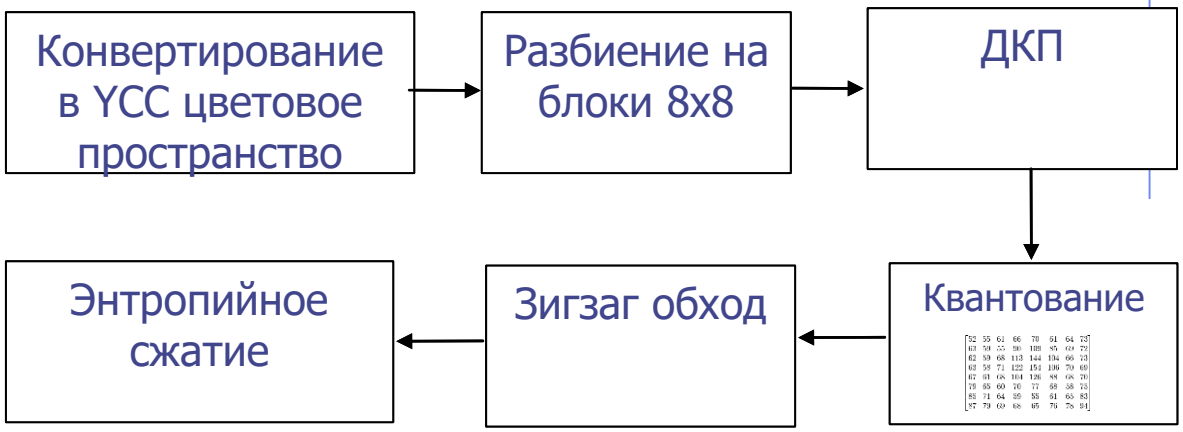
Шум не является белым, так как спектральные составляющие не распределены равномерно



Артефакты компрессии

В настоящее время используется много способов сжатия изображений с потерей информации. Все эти способы приводят к возникновению различных артефактов - дефектов. Одним из распространенных способов является JPEG.

Схема JPEG сжатия с потерями



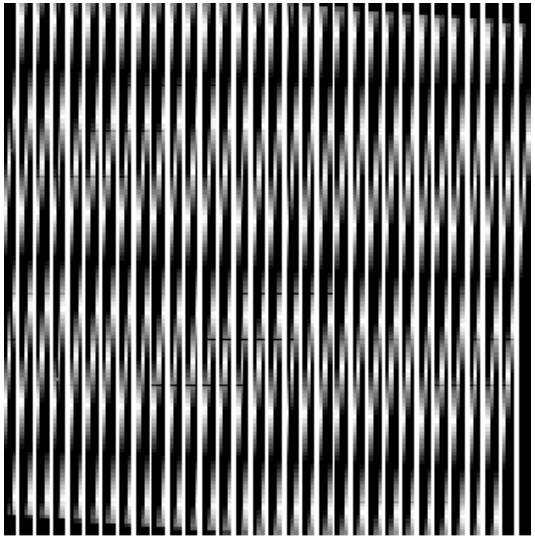
Блочные артефакты

Звон (*ringing*) или эффект Гиббса

Муар

В цифровых изображениях всегда присутствует эффект наложения спектров, который при определенных условиях принимает характер муара (*moire pattern*) – паразитной текстуры или регулярной помехи на изображении.

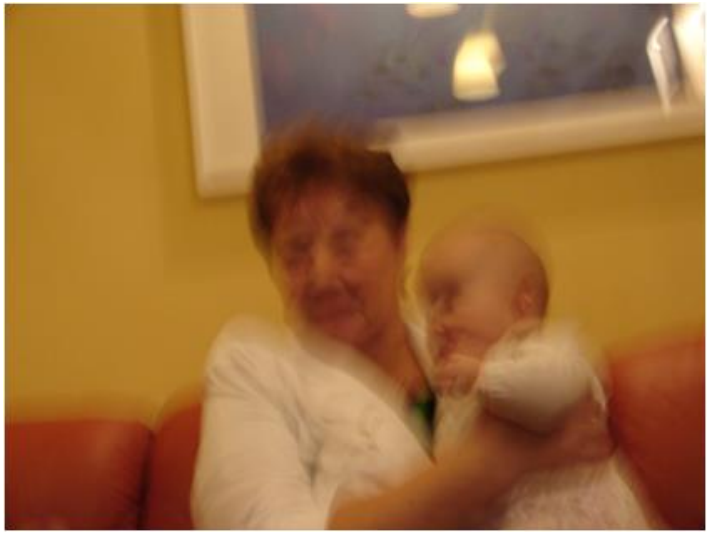
Возникновение муара иллюстрируют изображением двух наложенных друг на друга решеток, одной вертикальной, а другой повернутой на небольшой угол. Эффект муара виден в виде плоской интерференционной волны в вертикальном направлении.



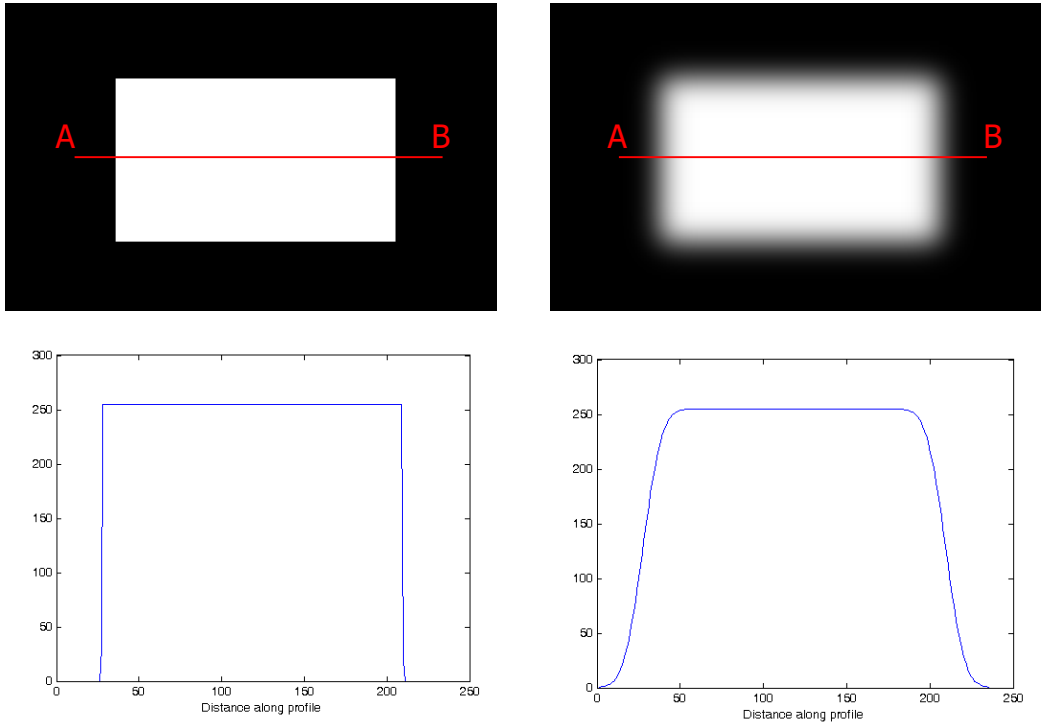
Муар может возникать, например, при сканировании растрированных при печати изображений.

Нечёткость/размытость

Нечёткость/размытость (*blurring*) изображения может возникать по ряду причин, например в следствие расфокусировки оптической системы аппаратуры регистрации изображений или в результате дрожания камеры (*shaking*)



Пример нечёткого изображения



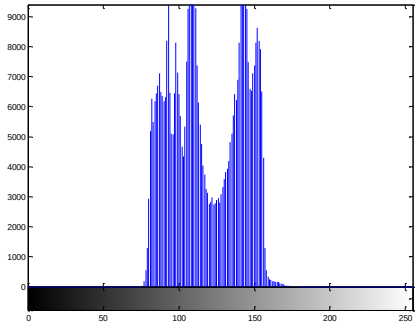
Профили яркости вдоль отрезка **AB**

Низкий контраст

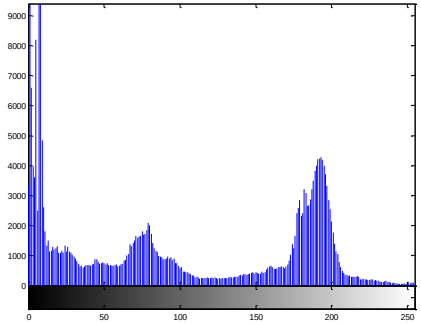
Контраст - отношение яркостей самой светлой и самой тёмной частей изображения. При низком контрасте детали изображения слабо различимы.

Глобально низкий контраст

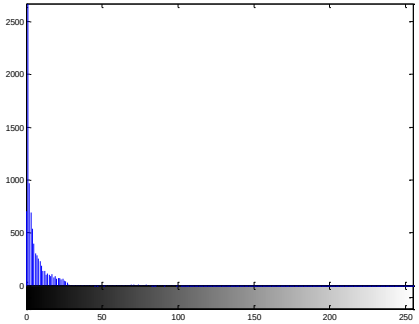
Локально низкий контраст на фото человека



“Узкая” гистограмма, не используется весь динамический диапазон



Глобальная гистограмма использует весь динамический диапазон



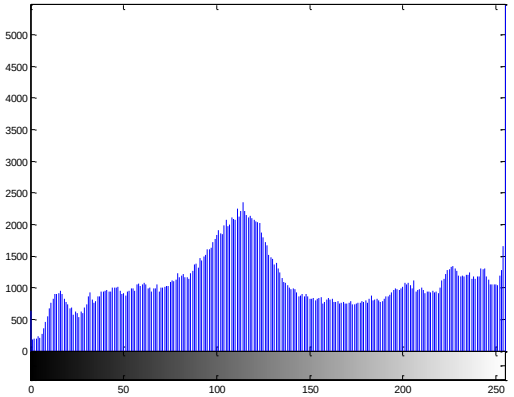
Но локальная гистограмма в области лица человека “узкая”

Искажения цветов

При регистрации цветных изображений могут возникать цветовые искажения (*color cast*, *color misbalance*) по причине ошибочной работы алгоритма баланса белого (*white balance*)

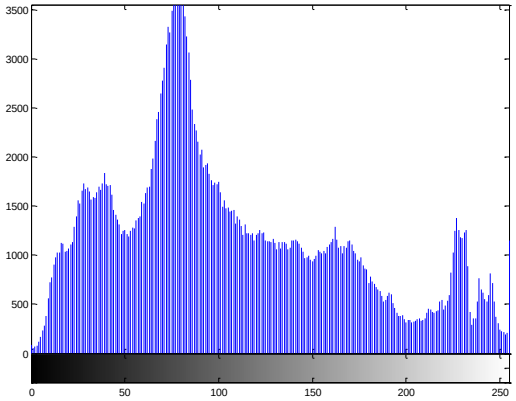
Yellow cast – слишком много жёлтого (или, что эквивалентно, мало синего) цвета

Гистограмма красного цвета



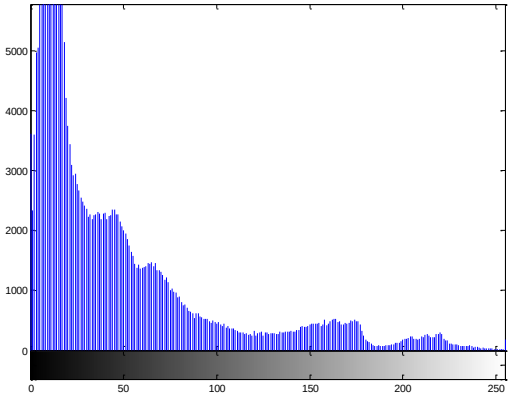
R

Гистограмма зеленого цвета



G

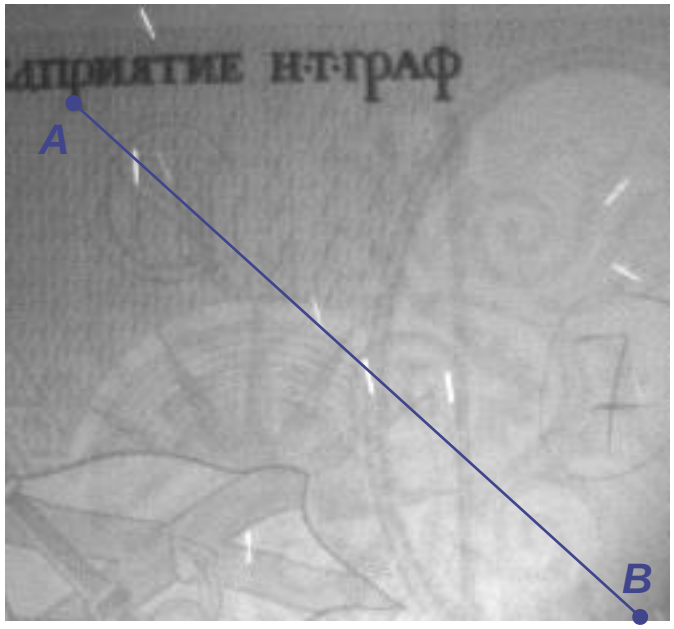
Гистограмма синего цвета



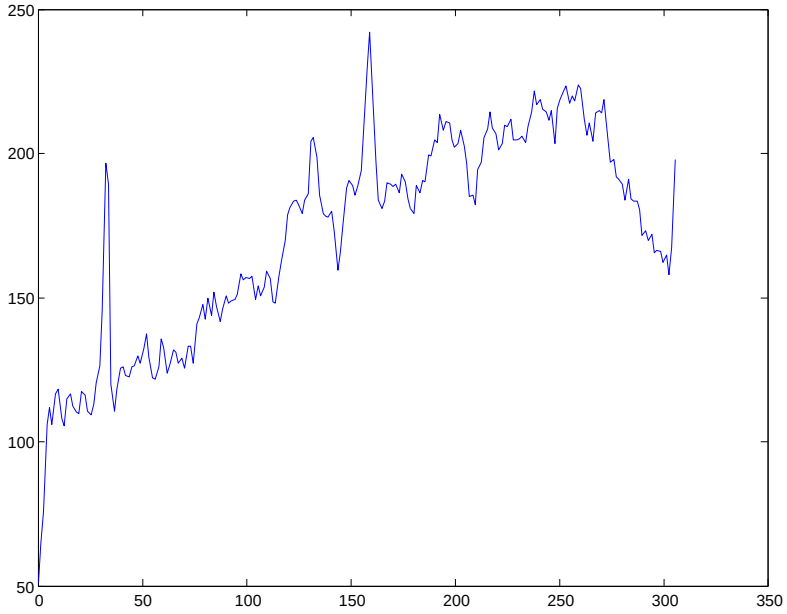
B

Неравномерный фон

Неравномерный фон (низкочастотный тренд/помеха) может мешать выделению на изображении объектов интереса с помощью алгоритмов сегментации.



На данном изображении образца ценной бумаги в лучах ультрафиолета неравномерный фон мешает выделять светящиеся нити.



Профиль яркости вдоль отрезка ***AB***

Классификация фильтров

Фильтр является линейным, если для него выполняется принцип суперпозиции:

$$f(aI_1 + bI_2) = af(I_1) + bf(I_2)$$

Т.е. сложный сигнал может быть разложен на более простые составляющие и к ним применен линейный оператор, результирующий отклик образуется из откликов составляющих. С другой стороны, единый фильтр можно получить из нескольких более простых.

Импульсной характеристикой фильтра называется его реакция на единичный импульс (дельта-функцию Дирака).

Линейные фильтры разделяют по виду импульсной характеристики на два класса:

- с конечной импульсной характеристикой (КИХ, finite impulse response, FIR), нерекурсивные фильтры:

$$y(n) = \sum_{i=0}^{P} b_i x(n-i)$$

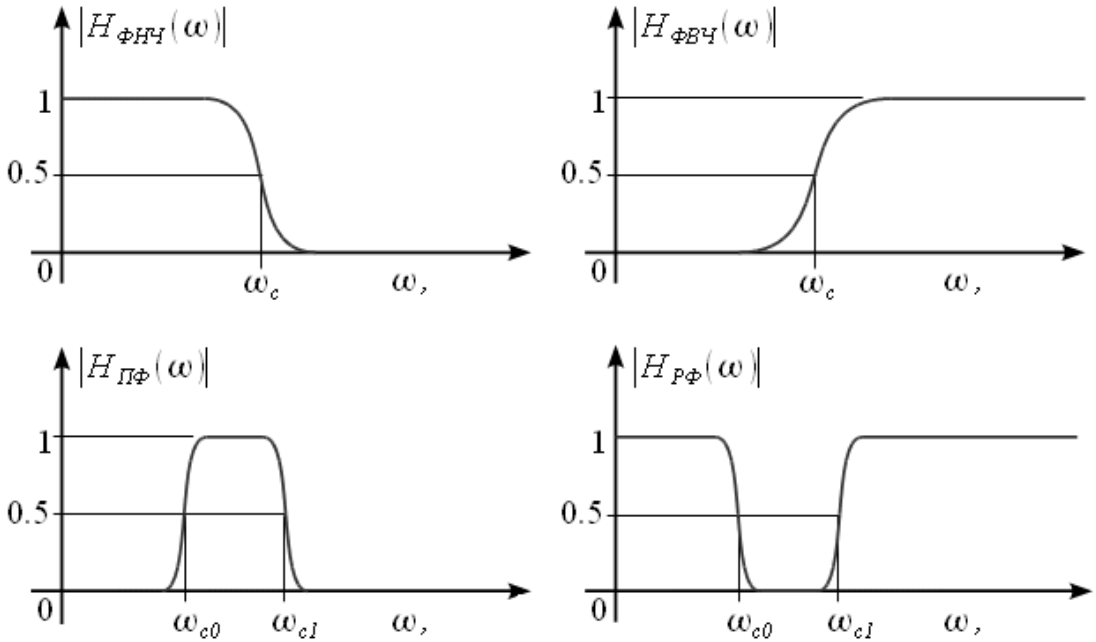
- с бесконечной импульсной характеристикой (БИХ, infinite impulse response, IIR), рекурсивные фильтры:

$$y(n) = \sum_{i=0}^{P} b_i x(n-i) - \sum_{k=1}^{Q} a_k y(n-k)$$

Амплитудно-частотные характеристики фильтров

По виду АЧХ фильтры подразделяются на:

- фильтр низких частот (ФНЧ, low-pass filter, LPF) – пропускает низкие частоты сигнала
- фильтр высоких частот (ФВЧ, high-pass filter, HPF) – пропускает высокие частоты сигнала
- полосовой фильтр (ПФ, band-pass filter) - пропускает ограниченную полосу частот сигнала
- режекторный фильтр (РФ, band-stop filter) - пропускает все частоты, кроме определённой полосы



Полосовые и режекторные фильтры могут быть сконструированы путём последовательного соединения ФНЧ и ФВЧ

Имена фильтрам даны по пропускаемым частотам (<u>не путать !!!</u>)

Свертка во временной области

Линейная свёртка (или просто свёртка, convolution) является одной из основных операций в цифровой обработке сигналов:

$$s(n) = \sum_{m=0}^{M-1} a(m)b(n-m)$$

Пример: a = [2, 1, 3, -1], b = [-1, 1, 2].

$a(m)$		2	1	3	-1	
$b(0-m)$		-1				
$b(1-m)$		1	-1			
$b(2-m)$		2	1	-1		
$b(3-m)$		2	1	-1		
$b(4-m)$		2	1			
$b(5-m)$		2				

$$s(0) = 2 \cdot (-1) = -2$$
$$s(1) = 2 \cdot 1 + 1 \cdot (-1) = 1$$
$$s(2) = 2 \cdot 2 + 1 \cdot 1 + 3 \cdot (-1) = 2$$
$$s(3) = 1 \cdot 2 + 3 \cdot 1 + (-1) \cdot (-1) = 6$$
$$s(4) = 3 \cdot 2 + (-1) \cdot 1 = 5$$
$$s(5) = (-1) \cdot 2 = -2$$

По краям сигнал a дополняют 0. В общем случае на краях результат свёртки может быть непригодным для дальнейшей обработки и обрезается.

Свойства свертки

Свертка является линейным оператором, так как для неё выполняется принцип суперпозиции: $H * (aI_1 + bI_2) = a(H * I_1) + b(H * I_2)$

Коммутативность: $I * H = H * I$

Ассоциативность: $I * (H * M) = (I * H) * M$

Дистрибутивность: $I * (H + M) = I * H + I * M$

Свёртку можно рассматривать как взвешенную сумму.

Свёртка реализует КИХ фильтр, элементы *b(i)* называют коэффициентами фильтра или ядром свёртки (convolution kernel).

Помимо линейной свёртки существует циклическая (круговая) свёртка, при которой количество отсчётов в сигнале и ядре одинаковое, элементы ядра циклически переставляются.

Свертка в пространственной области

Свертка в пространственной области выполняется для всех пикселов изображения *I* нерекурсивно:

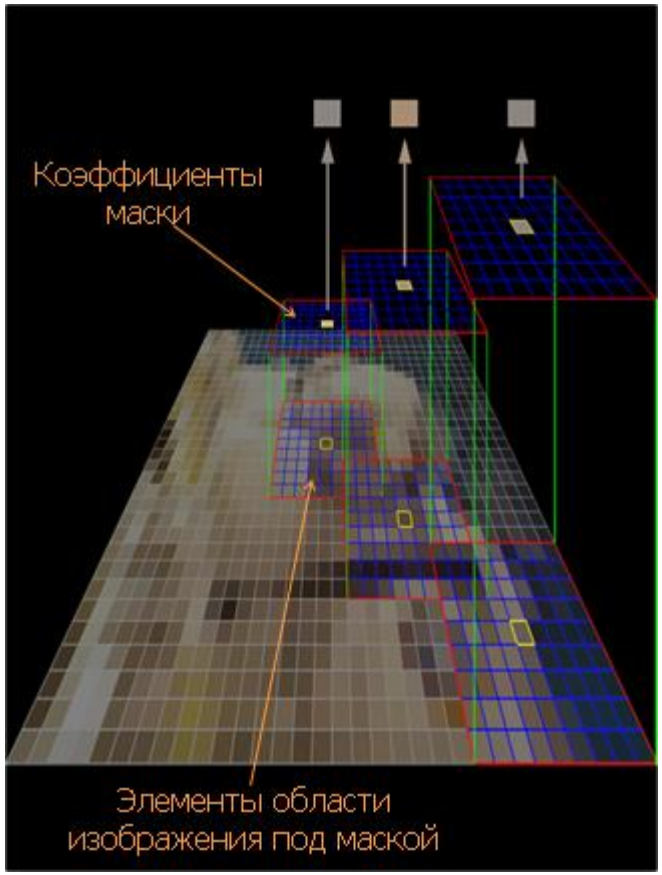
$$I'(r,c)=\sum_{i=1}^{K}\sum_{j=1}^{L} I\left(r+i-\frac{k+1}{2}, c+j-\frac{L+1}{2}\right)\times H(i,j)$$

H(i,j) называют ядром свертки, коэффициентами фильтра или маской. Размер ядра *K*x*L*.

$$I=\begin{bmatrix}5&7&8&1&3\\3&2&4&1&3\\6&4&0&1&1\\1&3&1&2&2\end{bmatrix}\quad I=\begin{bmatrix}5&7&8&1&3\\3&2&4&1&3\\6&4&0&1&1\\1&3&1&2&2\end{bmatrix}\quad I=\begin{bmatrix}5&7&8&1&3\\3&2&4&1&3\\6&4&0&1&1\\1&3&1&2&2\end{bmatrix}$$

$$H=\frac{1}{5}\begin{bmatrix}0&1&0\\1&1&1\\0&1&0\end{bmatrix}\qquad I'=\begin{bmatrix}?&?&?&?&?\\?&4&3&2&?\\?&3&2&1&?\\?&?&?&?&?\end{bmatrix}$$

Ядро «помещается сверху» изображения. Соответствующие друг другу элементы изображения и ядра перемножаются. Результаты произведений суммируются. Сумма записывается в новое изображение на место центрального элемента ядра.



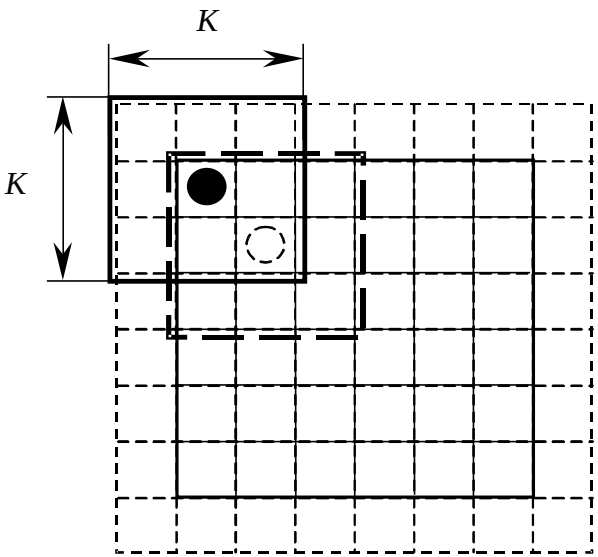
Обработка граничных строк и столбцов

Некоторая неопределенность существует с пикселами *K/2* с каждой стороны граничных сток и столбцов, где *K* – размер ядра свертки.

Используют один из следующих подходов:

• фильтрацию не выполняют для *K/2* с каждой стороны граничных сток и столбцов, и результирующее изображение получается на *K-1* строк и столбцов меньше

• фильтрацию не выполняют для *K/2* с каждой стороны граничных сток и столбцов, в результирующее изображение копируют значения этих строк и столбцов без изменений, результирующее изображение имеет тот же размер, что и исходное;



• до фильтрации дополняют исходное изображение с каждой стороны *K/2* строками и столбцами, при этом пикселы дополнительных строк и столбцов получают:

- с помощью заполнения константным значением, как правило, нулем;
- повторением крайних строк и столбцов;
- симметричным отражением строк и столбцов от границ изображения.
- с помощью заполнения значениями, которые не изменяют производную на границе изображения.

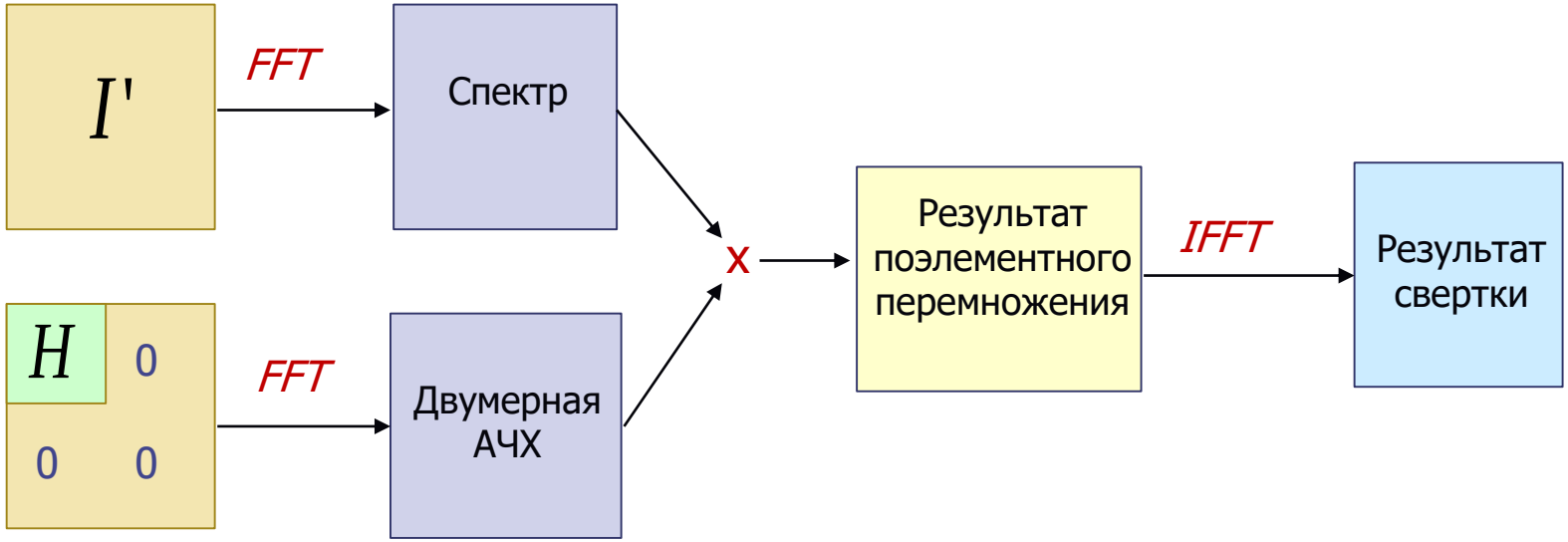
Пример свертки в пространственной области

Пример свертки в пространственной области для полутонового изображения 8 bpp. Для граничных строк и столбцов значения просто копируются.

```
void Convolution2D(const unsigned char* pSrc, unsigned char* pRes, int iWidth,
                   int iHeight, const int* pKernel, int iKernelSize ){
    int iHalf = iKernelSize / 2;
    memcpy( pRes, pSrc, sizeof(*pRes) * iWidth * iHeight );
        // копирование изображения (для полос по краям изображения)
    for ( int y = iHalf; y < iHeight - iHalf; ++y ) { // свертка
        for( int x = iHalf; x < iWidth - iHalf; ++x ){
            const int* pk = pKernel;
            const unsigned char* ps = &pSrc[(y - iHalf) * iWidth + x - iHalf];
            int iSum = 0;
            for ( int v = 0; v < iKernelSize; ++v ){
                for ( int u = 0; u < iKernelSize; ++u )
                    iSum += ps[u] * pk[u];

                pk += iKernelSize;  // Переход к следующей строкам
                ps += iWidth;
            }
            if ( iSum > 255 ) iSum = 255; // Контроль выхода из диапазона 8 bpp
            else
            if ( iSum < 0 ) iSum = 0;
            pRes[y * iWidth + x] = (unsigned char)iSum;
        }
    }
}
```

Свертка в частотной области



$$I * H = IFFT(FFT(I') \times FFT(H'))$$

I – исходное изображение
I' – изображение дополненное нулями до размера кратного степени двух
H – ядро свертки H' – ядро свертки, дополненное нулями до размера изображения I'

Сепарабельные фильтры

Некоторые линейные двумерные фильтры являются разделимыми (сепарабельными), т.е. двумерную фильтрацию можно заменить на две одномерные фильтрации по строкам, затем по столбцам. Например, сепарабельным является двумерный фильтр Гаусса.

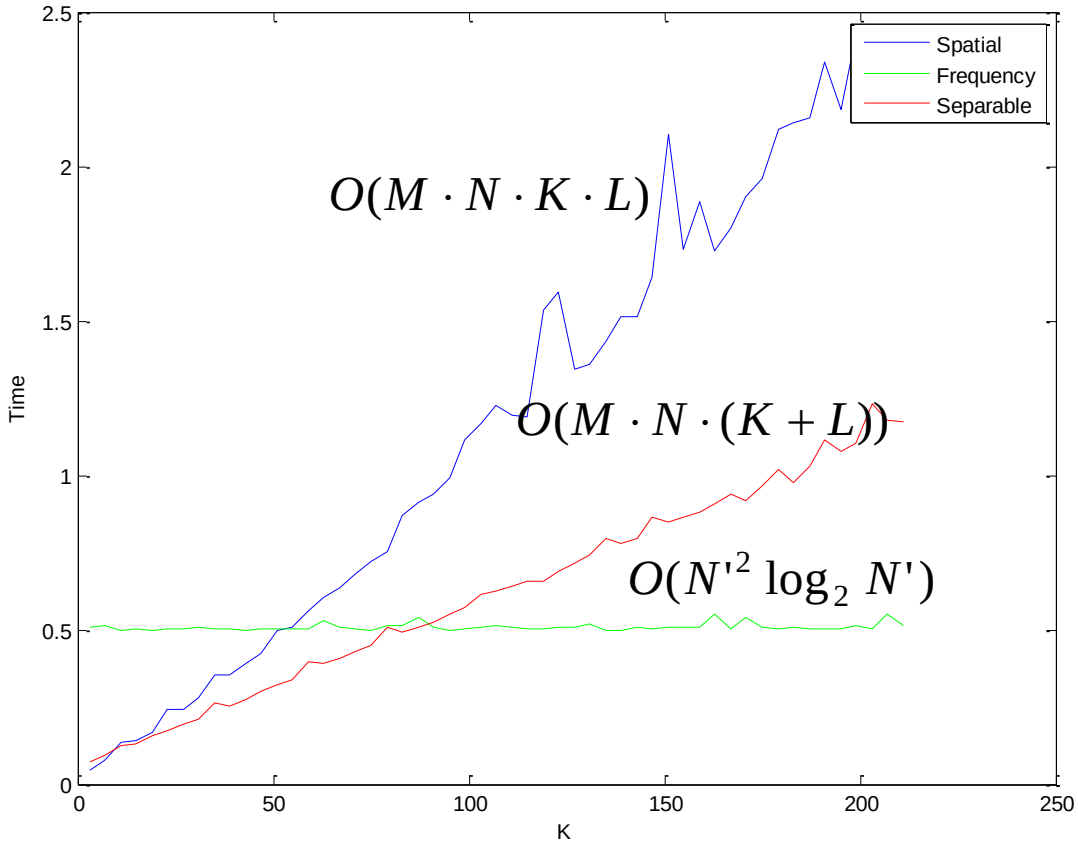
$$I_2 = G * I_1 = \sum_h \sum_k G(h,k) I_1(i-h, j-k) = \sum_h \sum_k e^{\frac{-h^2-k^2}{2\sigma^2}} I_1(i-h, j-k) =$$

$$= \sum_h e^{\frac{-h^2}{2\sigma^2}} \sum_k e^{\frac{-k^2}{2\sigma^2}} I_1(i-h, j-k)$$

Часто для других фильтров сепарабельность не доказана строго математически, но использование аналогичной обработки по строкам, затем по столбцам позволяет получить близкие у двумерному фильтру качественный и количественные результаты при уменьшении времени обработки.

Эффективность различных способов свертки

В зависимости от размера ядра свертки и вычислительной платформы эффективнее использовать тот или иной способ свертки. Для достаточно больших размеров ядер предпочтительнее использовать свертку в частотной области.



Для каждой конкретной компьютерной архитектуры и реализации различных способов свертки, соотношение скорости работы в зависимости от размера ядра могут различаться, но общий характер зависимостей остается похожим.